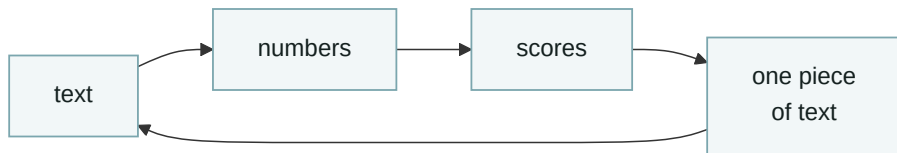


Chapter 1 — Substrate

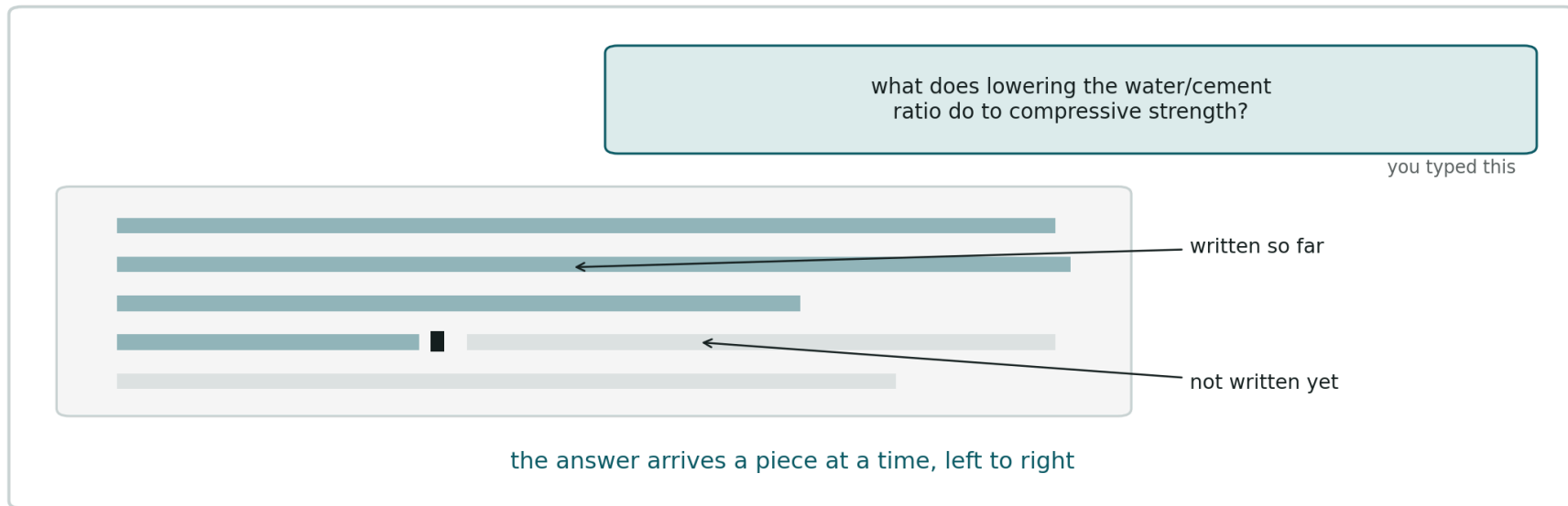
What a language model is, and what one run of it computes



Nothing in this chapter assumes you have seen any of it before.

The answer arrives a piece at a time

SCHEMATIC of the interface — capture a real screenshot before delivery



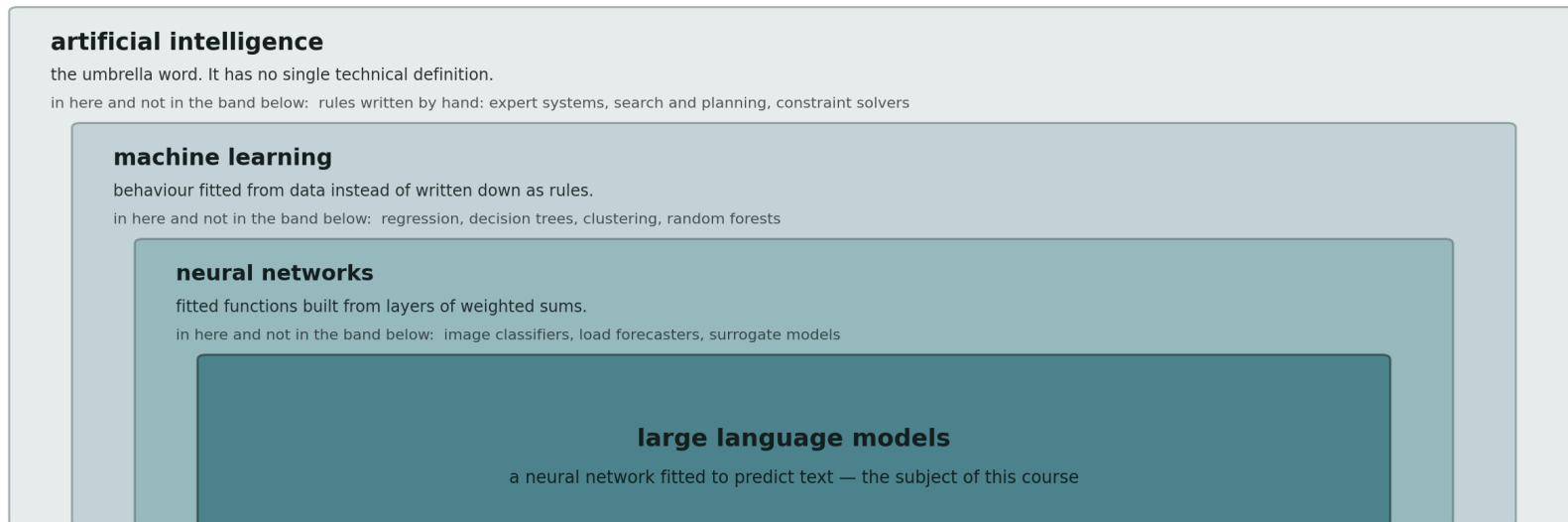
You type a question — the text you send is the prompt

It writes left to right, in pieces

This chapter what each piece costs to produce

Artificial intelligence names a family of methods, and language models are one branch of it

Every band contains the one inside it. Not all artificial intelligence is a language model.



Large language model — a neural network fitted to predict text. Shortened to LLM.

A model is a long list of numbers together with the program that does arithmetic with them

the parameters

one long list of numbers

-0.0412	0.9037	0.1188	-1.2740
0.0006	0.4451	-0.3319	0.7702
0.0925	-0.0187	1.1046	-0.6538

... and so on, counted in billions

the program

the arithmetic that uses them

1. read the numbers from the list
2. multiply and add them against the input, layer after layer
3. hand back one score per token

Both are files on a disk. Together they are the model.

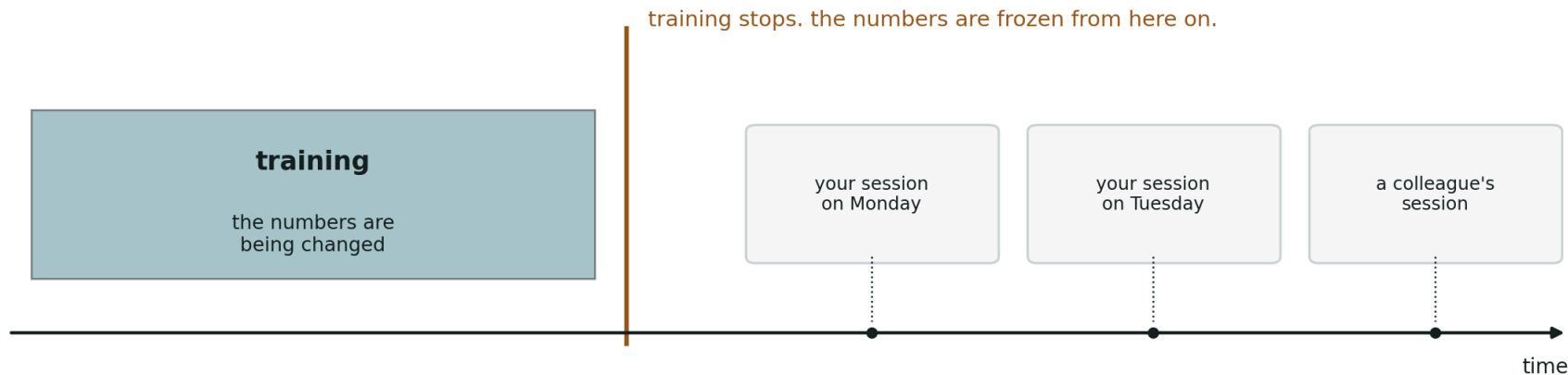
Parameters also called weights

Network the layered arithmetic

Size how many parameters

Training set those numbers and then stopped

Nothing to the right of the line changes anything to the left of it.



Training the numbers are being changed

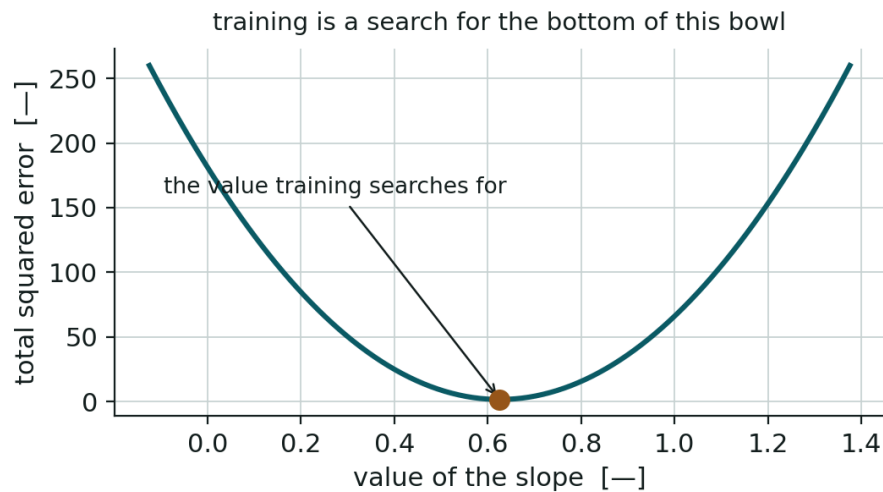
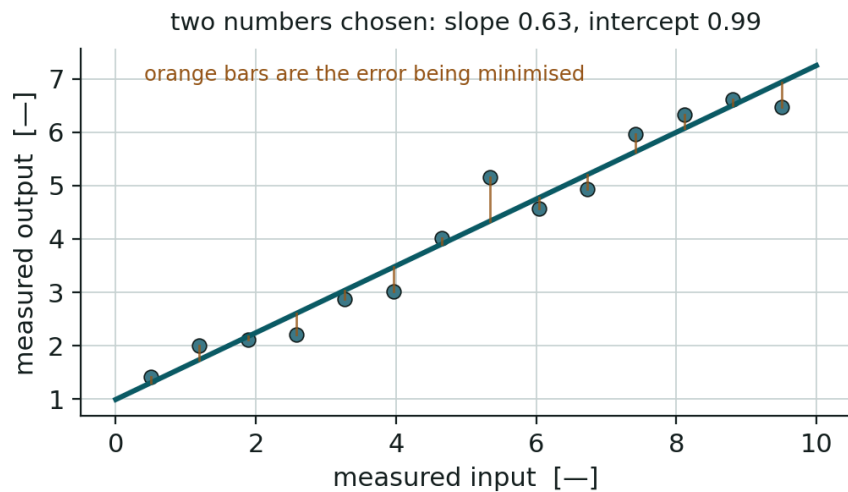
Inference running the finished numbers on
your text

Knowledge cutoff the date training stopped

Everything you will ever do with it happens to the right of the line. One **session** is one continuous conversation, from opening it to closing it.

Training chooses parameter values that make the total error smallest

A language model does exactly this with billions of numbers instead of two, and the data is text.



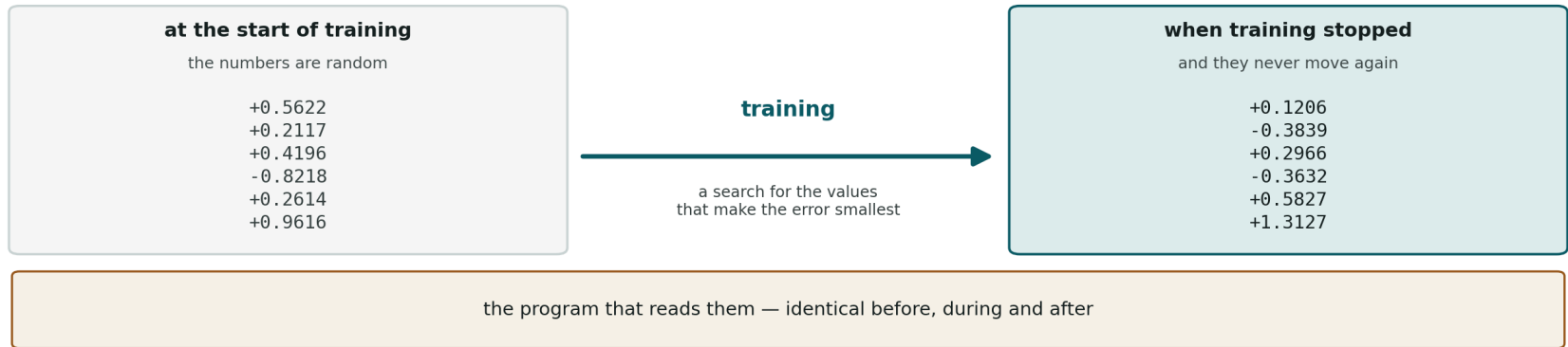
Two parameters slope and intercept

One error summed over the data

One search for the values at the minimum

Neural network and learning are borrowed names for a search over numbers

ILLUSTRATIVE values, drawn from a fixed seed. The movement is the point, not the numbers.



Neural network layers of multiply-and-add,
named after a biological analogy

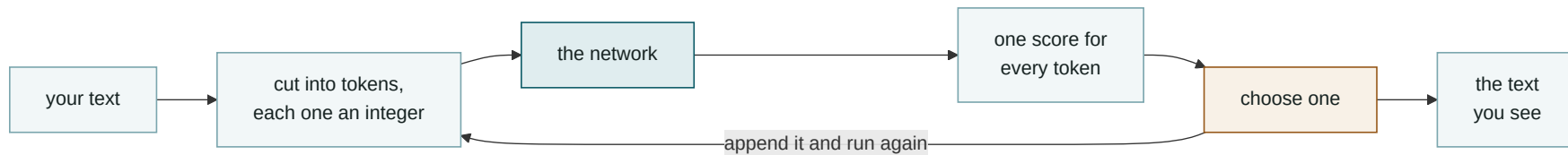
Learning a search for values that lower the
error

The brain comparison correlational; the
appendix carries what it establishes

[V] Schrimpf, M., Blank, I. A., Tuckute, G., et al., 2021. *PNAS* 118(45):e2105646118, DOI 10.1073/pnas.2105646118.

[V] Hadidi, N., Fegghi, E., Song, B. H., Blank, I. A. & Kao, J. C., 2026. *Nature Communications* 17(1):5769, DOI 10.1038/s41467-026-72253-7.

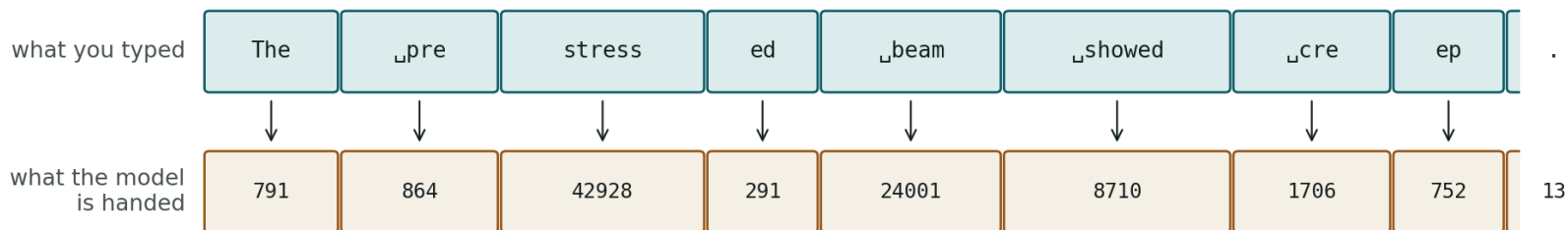
The whole path runs from text to numbers to scores to one piece of text, and then repeats



Token — a chunk of characters from a fixed list, defined on the next slide. Every box here gets its own slide.

A token is a chunk of characters drawn from a fixed list that was settled before training

SCHEMATIC — no tokeniser was run. Capture a real split before delivery.



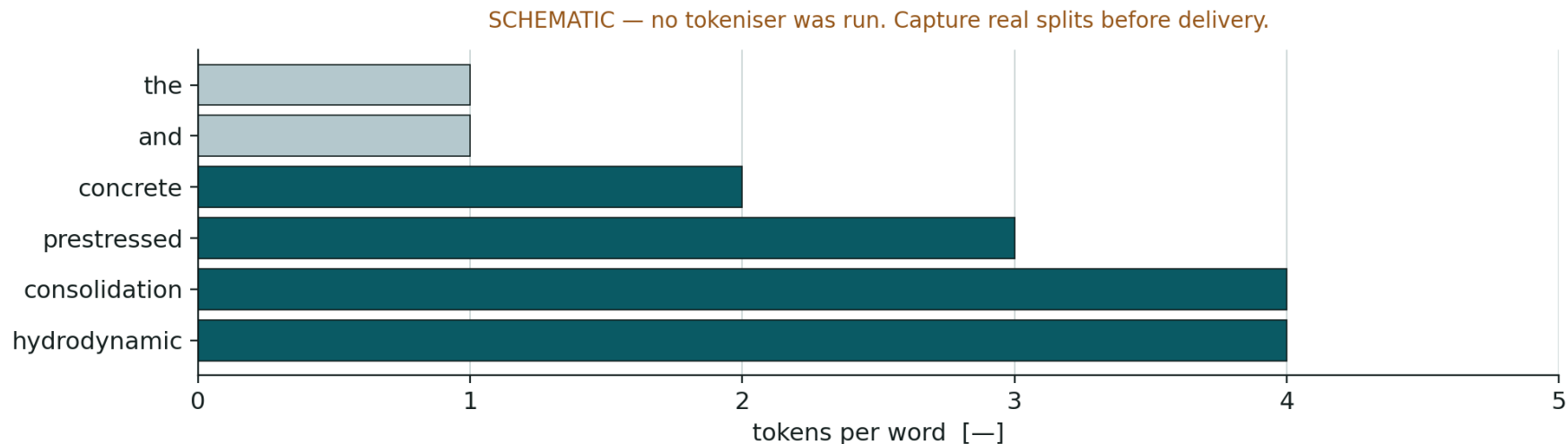
pre marks a leading space, which belongs to the token. Nine tokens for five words.

Fixed list the vocabulary — tens of thousands of entries

Roughly 3–4 characters each, in English

The network sees integers never letters

Technical terms are split into more tokens than common words



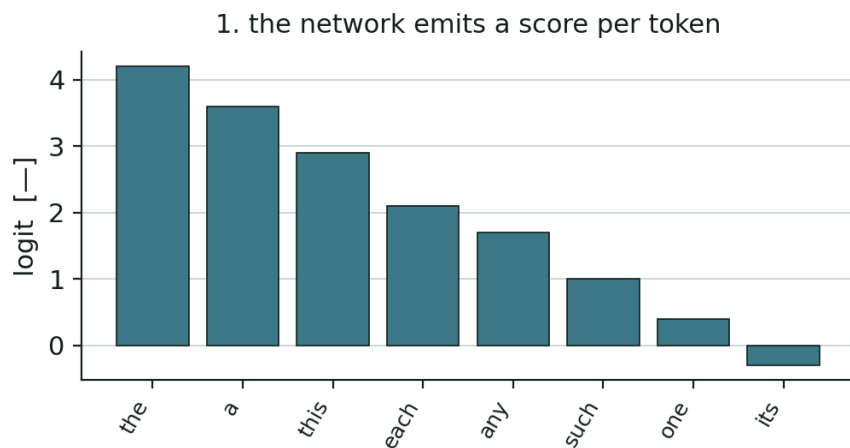
Billed per token

Context limit counted in tokens

Speed quoted per token

The network returns one score for every token in the list, because scoring is the only operation it performs

illustrative values — the transform is exact



The raw scores are called **logits**. A higher score means the token fits the text so far better, according to the fitted numbers. One run of the network over the window is one **pass**.

Attention is the weighted sum that decides which earlier tokens the next score leans on

ILLUSTRATIVE weights — drawn to show the operation, not read out of a model.

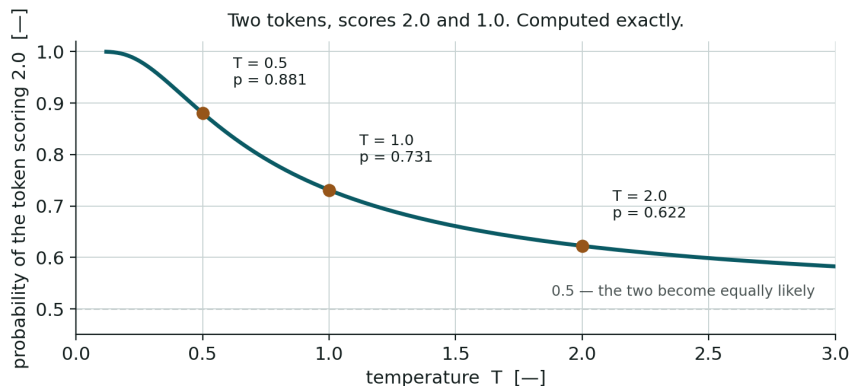


Thicker arc, larger share of the weighted sum. The weights are recomputed for every new token.

The paper defines it as a **weighted sum**, in which the weight on each earlier position is computed by comparing that position against the one being scored. **Transformer** is the name given to the arrangement of arithmetic built out of that operation. The word is borrowed from biology as well, and the paper that introduced the operation is about machine translation.

Softmax converts the scores into probabilities that add up to one

$$p_i = \frac{\exp\left(\frac{\overbrace{z_i}^{\text{score for token } i}}{\underbrace{T}_{\text{temperature}}}\right)}{\underbrace{\sum_j \exp(z_j/T)}_{\text{sum over every token, so the result adds to 1}}}$$



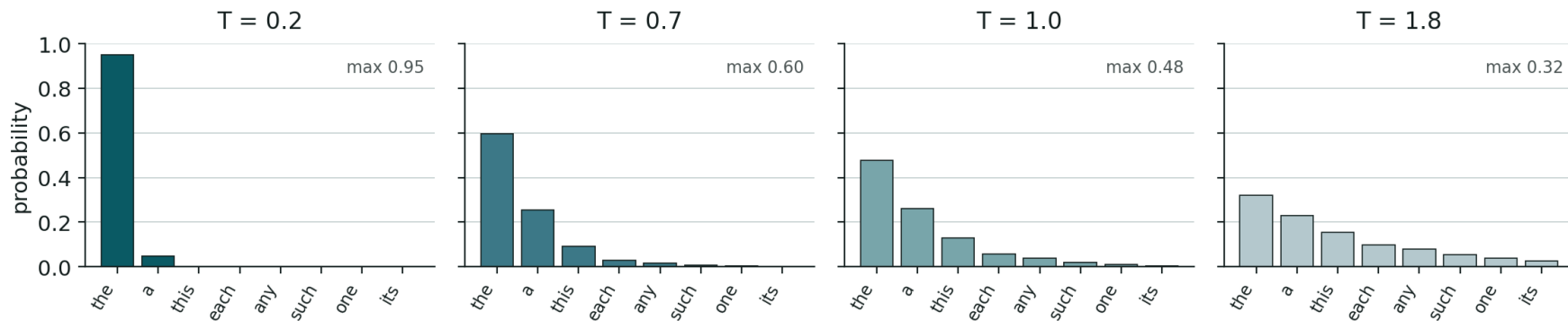
Worked case — two tokens, scores 2.0 and 1.0, at $T = 1$

$$p_1 = \frac{e^{2.0}}{e^{2.0} + e^{1.0}} = \frac{7.389}{7.389 + 2.718} = 0.731$$

All quantities dimensionless. z and T carry no units, and p is a probability.

Temperature sets how sharply the highest score wins

One set of logits, four temperatures — logits illustrative, softmax exact

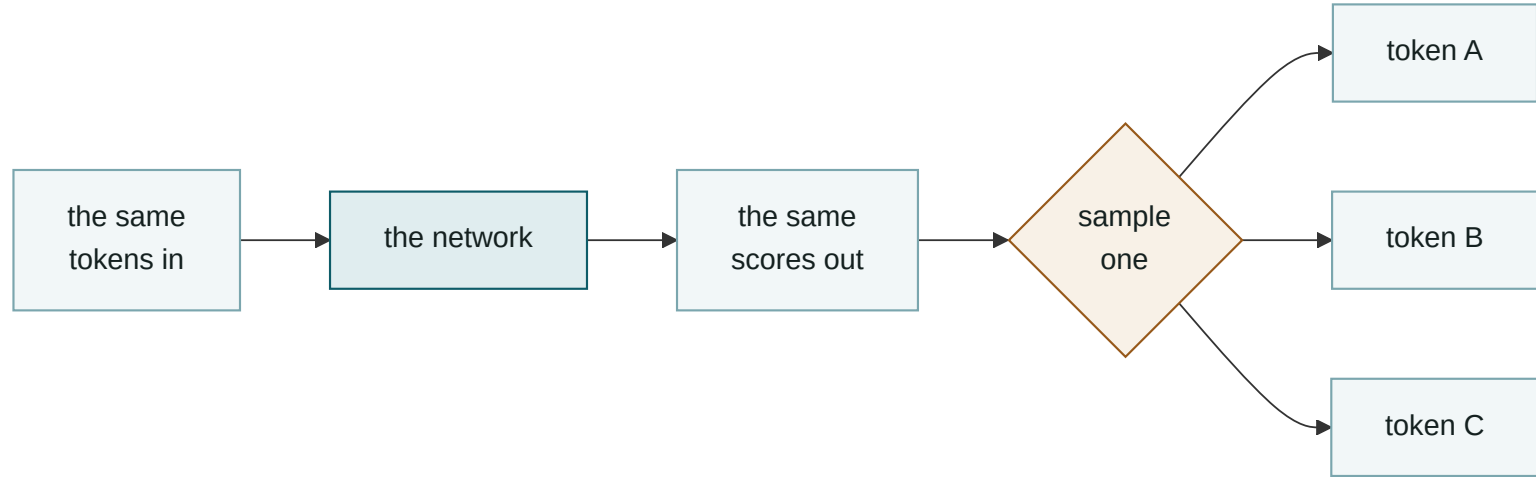


Low T one token takes almost all the probability

T = 1 the scores are used as they are

High T the probability spreads across many tokens

Two runs of the same prompt give the same scores and can still give different answers

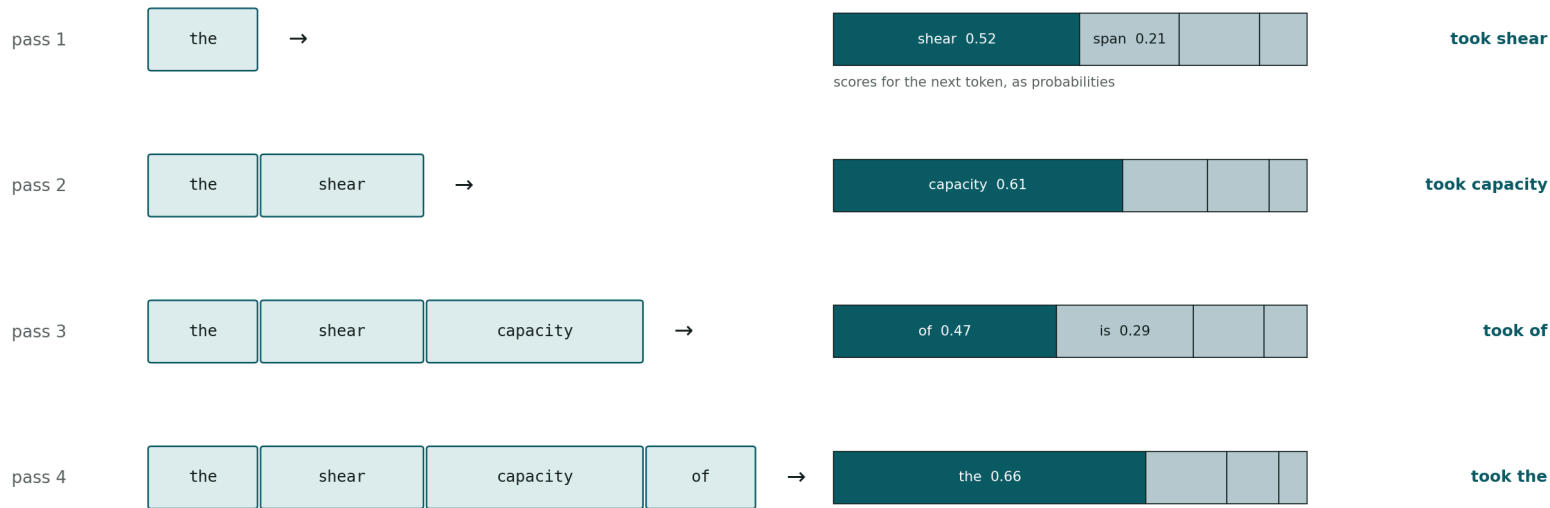


The network fixed numbers, fixed arithmetic, one answer

The sampler draws from the probabilities, so it can land elsewhere

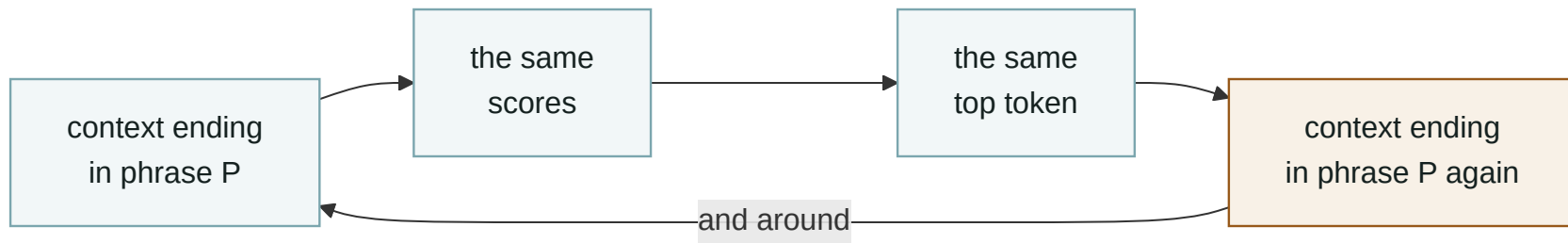
Each chosen token is appended and the whole program runs again on the longer text

ILLUSTRATIVE — the same frozen numbers are read on every pass. Only the text handed to them has grown.



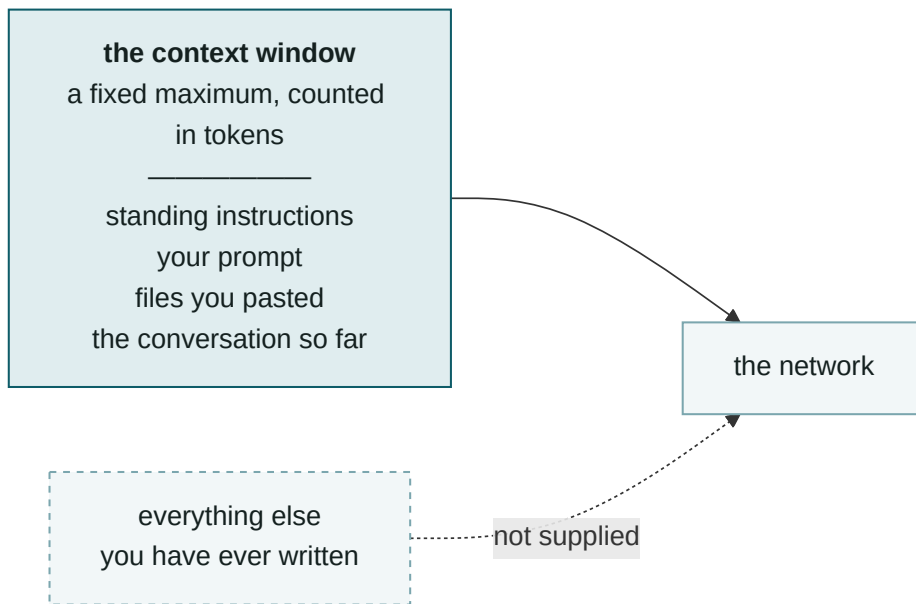
Four passes here, four scores over the whole token list, four choices. A paragraph is a few hundred of these.

Taking the highest-scoring token every time makes the text fall into a loop



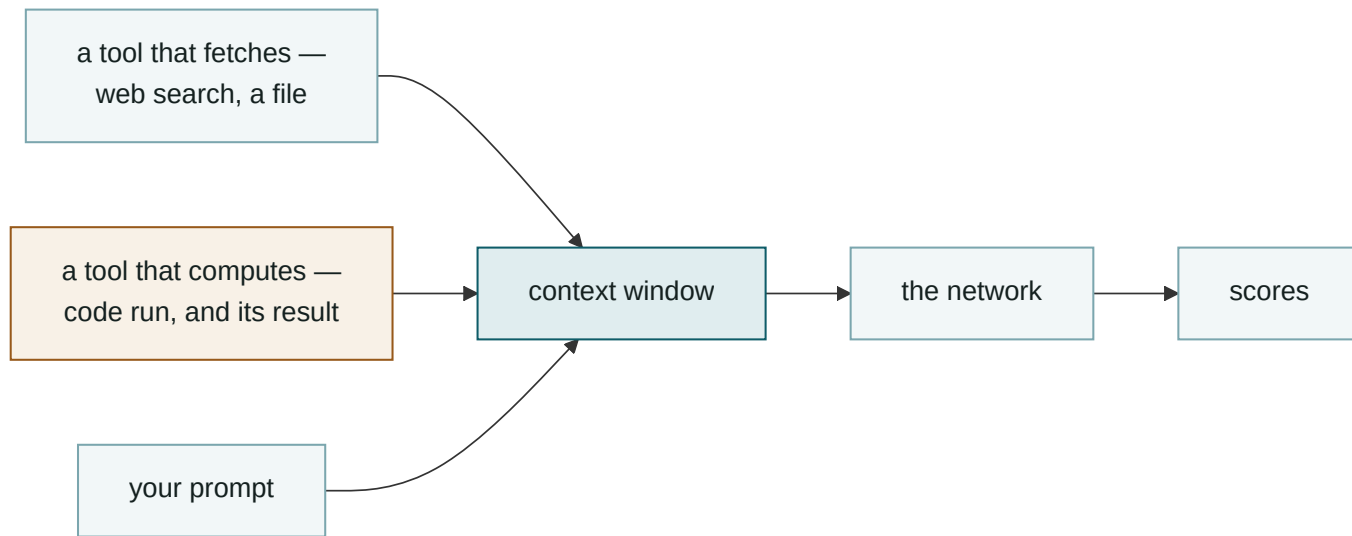
Taking the top-scoring token every time is called **greedy** selection. Same numbers, same input, same output — so once a phrase recurs, the context that produced it recurs with it, and the top-scoring token is the same one again.

The network is given the tokens in the context window and nothing else



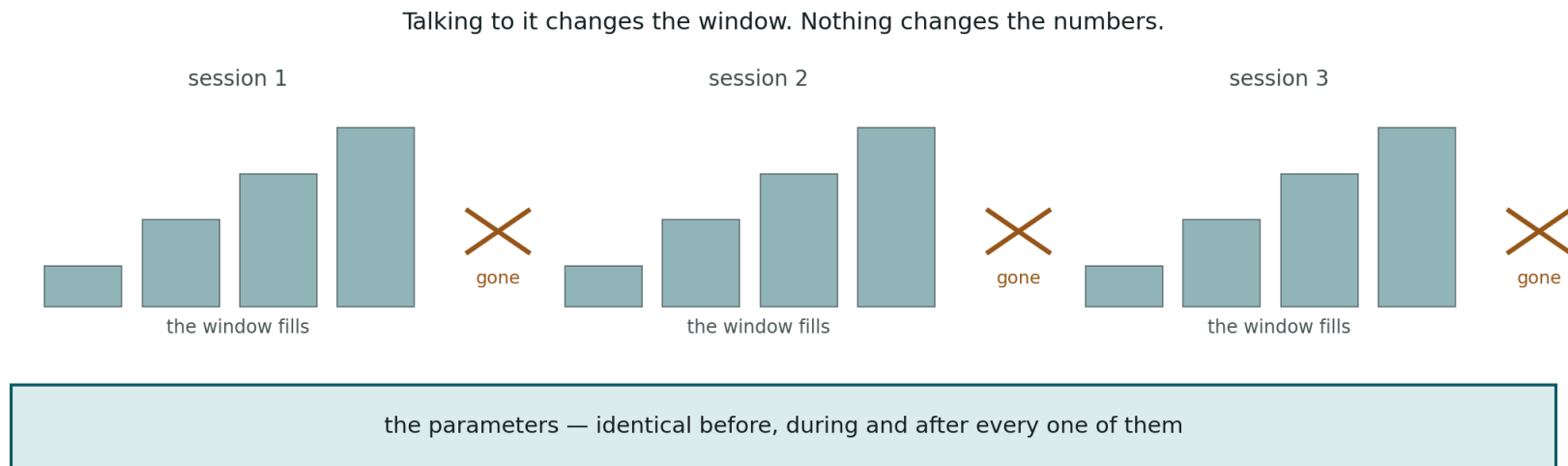
The window is the input to one pass. It holds tokens, it has a maximum size, and it is re-read in full on every pass. **Standing instructions** are text placed in the window at the start of every session, which Chapter 10 is about.

Tools add text to the window and can run code beside the network, and the scoring step is unchanged



A tool is a program the system can run alongside the network. It fetches or computes, its result is written into the window as tokens, and the pass that follows is the one already described.

The window is discarded when the session ends and the numbers are left as they were

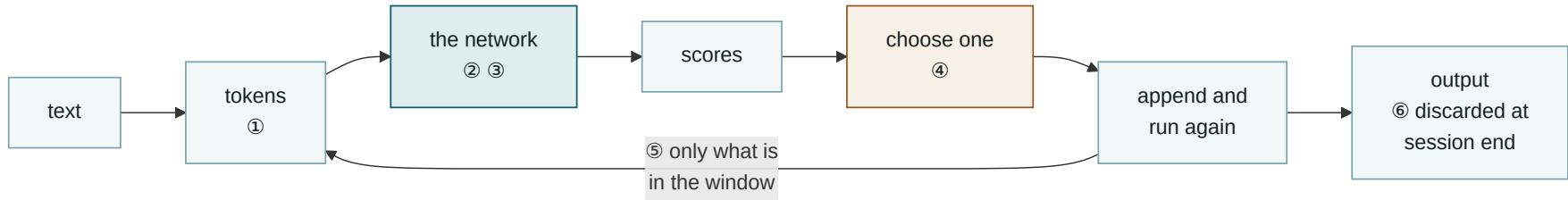


Discarded the window, the conversation, anything you pasted

Unchanged the parameters, in every session

When the system appears to remember your project, something outside the network put that text back into the window.

Six facts, and where each one is used later



	Used in
① Text is cut into tokens from a fixed list	Ch 7 cost, Ch 13 billing
② The parameters were fixed by training and are only read	Ch 2 formation, Ch 10 memory
③ The network returns one score per token , every pass	Ch 5 hallucination
④ The scores are fixed; the choice among them is sampled	Ch 4 measurement
⑤ It conditions only on the context window	Ch 5 degradation, Ch 6 injection
⑥ The window is discarded at the end of the session	Ch 9, Ch 10